

Backend Technology Stack

Backend Configuration :

```
const { Pool } = require("pg");
require("dotenv").config();

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
});

pool.connect()
  .then(() => {
    console.log("✅ PostgreSQL Connected Successfully");
  })
  .catch((err) => {
    console.log("❌ Database Connection Failed");
    console.error(err.message);
  });

module.exports = pool;
```

Database:

HTTP **http://localhost:5000/api/auth/register**

POST **http://localhost:5000/api/auth/register**

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** ☐

```
1 {
2   "name": "Nazia Neha",
3   "email": "neha@gmail.com",
4   "password": "123456",
5   "phone": "01712345678",
6   "user_type": "donor"
7 }
```

Body Cookies Headers (8) Test Results 201 Created

Pretty Raw Preview Visualize JSON ☐

```
1 {
2   "success": true,
3   "message": "User Registered Successfully",
4   "user": {
5     "user_id": 1,
6     "name": "Nazia Neha",
7     "email": "neha@gmail.com",
8     "user_type": "donor"
9   }
10 }
```

```
PS F:\blood-donation-management-system\backend> node server.js
Server is running on Port 5000
PostgreSQL Connected Successfully
```

Implemented APIs :

Data Output Messages Notifications					
Showing rows: 1 to 1 Page No: 1 of 1					
		name character varying (100)	email character varying (100)	password_hash character varying (255)	phone character varying (20)
1	1	Nazia Neha	neha@gmail.com	\$2b\$10\$/PihFR3GKjqCuEssNX2PH0t85luVFc8/zYgHoaVCYTKGTsn17o...	01712345678

Showing rows: 1 to 1 Page No: 1 of 1			
phone character varying (20)	user_type character varying (20)	created_at timestamp without time zone	
01712345678	donor	2026-06-29 22:54:56.419983	

Authentication Process:

```
const pool = require("../config/db");
const bcrypt = require("bcrypt");

const register = async (req, res) => {
  try {
    const { name, email, password, phone, user_type } = req.body;

    // Check if email already exists
    const existingUser = await pool.query(
      "SELECT * FROM users WHERE email = $1",
      [email]
    );

    if (existingUser.rows.length > 0) {
      return res.status(400).json({
        success: false,
        message: "Email already exists"
      });
    }

    // Password Hash
    const hashedPassword = await bcrypt.hash(password, 10);

    // Insert User
    const result = await pool.query(
```

```

        `INSERT INTO users(name,email,password_hash,phone,user_type)
        VALUES($1,$2,$3,$4,$5)
        RETURNING user_id,name,email,user_type`,
        [name, email, hashedPassword, phone, user_type]
    );

    res.status(201).json({
        success: true,
        message: "User Registered Successfully",
        user: result.rows[0]
    });

} catch (error) {
    console.error(error);

    res.status(500).json({
        success: false,
        message: "Server Error"
    });
}
};

module.exports = { register };

```

Router :

```

const express = require("express");
const router = express.Router();

const { register } = require("../controllers/authController");

router.post("/register", register);

module.exports = router;

```

Server.js :

```

const express = require("express");
const cors = require("cors");
require("dotenv").config();

```

```

require("./config/db");

const authRoutes = require("./routes/authRoutes");

const app = express();

app.use(cors());
app.use(express.json());

// Default Route
app.get("/", (req, res) => {
    res.send("AI Blood Donation Management System Backend is Running...");
});

// Authentication Routes
app.use("/api/auth", authRoutes);

// Start Server
const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
    console.log(`🚀 Server is running on Port ${PORT}`);
});

```

Backend Folder Structure :

